

NAME:S.DHARSHINI

**College: v.s.b college of engineering
technical compass,coimbatore**

Email : ddharshu457@gmail.com

PYTHON MINI PROJECT

BLOG PLATFORM USING FLASK

Objective

The objective of this project is to develop a simple Blog Platform using the Flask framework in Python. The application allows users to create, view, edit, and delete blog posts through a web interface. This project helps in understanding web development concepts such as routing, templates, forms, and database operations.

Introduction

A Blog Platform is a web application where users can publish articles, share ideas, and manage blog posts. Blogging websites are widely used for education, news, personal experiences, and business communication.

This project is developed using the Flask framework and SQLite database. It demonstrates the basics of web development in Python, including routing, HTML templates, database connectivity, CRUD (Create, Read, Update, Delete) operations, and user interaction.

Problem Statement

Managing blog posts manually is time-consuming and inefficient. A web-based blog platform allows users to create, edit, view, and delete posts easily through an organized interface. This project provides a simple and efficient blogging system using Python and Flask.

Scope of the Project

- Create new blog posts.
 - View all published posts.
 - Edit existing posts.
 - Delete unwanted posts.
 - Store blog data using SQLite.
 - Learn web development using Flask.
-

Software Requirements

Hardware Requirements

- Computer or Laptop
- Minimum 2 GB RAM
- Intel i3 Processor or above

Software Requirements

- Python 3.x
 - Flask Framework
 - SQLite Database
 - Visual Studio Code / PyCharm
 - Web Browser (Chrome, Edge, Firefox)
-

Modules Used

- **Flask** – Web application framework.
- **sqlite3** – Database management.
- **render_template()** – Displays HTML pages.
- **request** – Handles user input.
- **redirect()** – Redirects users to another page.

- **url_for()** – Generates URLs for routes.

Install Flask using:

```
pip install flask
```

Algorithm

1. Start the application.
 2. Create a Flask application.
 3. Connect to the SQLite database.
 4. Display all available blog posts.
 5. Allow users to create a new post.
 6. Save the post in the database.
 7. Allow users to edit or delete existing posts.
 8. Update the database accordingly.
 9. Display the updated list of blog posts.
 10. Stop the application.
-

Python Program

```
from flask import Flask, render_template_string, request, redirect
import sqlite3
```

```
app = Flask(__name__)
```

```
def init_db():
    conn = sqlite3.connect("blog.db")
    cur = conn.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS posts(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            title TEXT,
            content TEXT
        )
    """)
    conn.commit()
    conn.close()
```

```
@app.route("/")
def home():
```

```

conn = sqlite3.connect("blog.db")
cur = conn.cursor()
cur.execute("SELECT * FROM posts")
posts = cur.fetchall()
conn.close()

html = ""
<h1>Simple Blog Platform</h1>
<a href="/add">Add New Post</a><br><br>

{% for post in posts %}
  <h2>{{post[1]}}</h2>
  <p>{{post[2]}}</p>
  <a href="/delete/{{post[0]}}">Delete</a>
  <hr>
{% endfor %}
""

return render_template_string(html, posts=posts)

```

```

@app.route("/add", methods=["GET","POST"])
def add():
    if request.method == "POST":
        title = request.form["title"]
        content = request.form["content"]

        conn = sqlite3.connect("blog.db")
        cur = conn.cursor()
        cur.execute(
            "INSERT INTO posts(title,content) VALUES(?,?)",
            (title,content)
        )
        conn.commit()
        conn.close()

        return redirect("/")

    return ""
<h2>Add Blog Post</h2>

<form method="post">
  Title:<br>
  <input type="text" name="title"><br><br>

  Content:<br>
  <textarea name="content"></textarea><br><br>

  <input type="submit" value="Publish">

```

```
</form>
""
```

```
@app.route("/delete/<int:id>")
def delete(id):

    conn = sqlite3.connect("blog.db")
    cur = conn.cursor()
    cur.execute("DELETE FROM posts WHERE id=?", (id,))
    conn.commit()
    conn.close()

    return redirect("/")

if __name__ == "__main__":
    init_db()
    app.run(debug=True)
```

Sample Output

Simple Blog Platform

Python Programming

Flask is a lightweight Python web framework.

Delete

Machine Learning

Machine Learning is the future of AI.

Delete

Add New Post

Advantages

- Simple and user-friendly interface.
 - Stores blog posts in a database.
 - Easy to create, view, and delete posts.
 - Demonstrates CRUD operations.
 - Helps beginners learn Flask web development.
-

Limitations

- No user login or authentication.
 - Uses SQLite, suitable for small applications.
 - Does not support image uploads.
 - Basic user interface.
-

Future Enhancements

- User Registration and Login.
 - Rich Text Editor.
 - Image Upload Feature.
 - Search Functionality.
 - Categories and Tags.
 - Comment System.
 - Responsive User Interface.
 - Admin Dashboard.
 - Deploy the application online.
-

Conclusion

The **Blog Platform Using Flask** project demonstrates how Python can be used to build dynamic web applications. The project allows users to create, view, and delete blog posts using the Flask framework and SQLite database. It introduces important web development concepts such as routing, templates, forms, CRUD operations, and database connectivity. This project serves as a strong foundation for developing advanced blogging and content management systems.